

# Lesson 9 Worksheet: Final Project Planning – ANSWER KEY

## Part A: System Design

### 1. Diagram answers:

- `compute_manhattan_path()` returns: A list of (row, col) tuples representing each position along the path
- What two things does `drive_path()` do at each step?
  - Turn the robot to face the needed heading (using `turn_to`)
  - Follow the line to the next intersection (using `track_until_cross`, clearing first if going straight)
- `drive_path()` returns: (position, heading)

### 2. After `drive_path()` returns, what must you do with its return value?

You must reassign `position, heading = drive_path(path, position, heading)`. Without this, `drive_path()` computes the new position and heading internally but the main program's own `position / heading` variables never update, so the next `compute_manhattan_path()` call would start from the wrong place.

### 3. Flow diagram blanks:

- `position = (0, 0)`
- `heading = 0`
- Compute **path** using `compute_manhattan_path`
- Drive the path, capturing new **position** and **heading**

## Part B: Destination Planning (Example Answer)

Note: Student answers will vary. This example uses destinations [(2, 0), (2, 3), (0, 3), (0, 0)].

| Order | Destination | Description              |
|-------|-------------|--------------------------|
| Start | (0, 0)      | Home base                |
| 1     | (2, 0)      | Bottom-left area         |
| 2     | (2, 3)      | Bottom-right corner area |
| 3     | (0, 3)      | Top-right corner         |
| 4     | (0, 0)      | Return home              |

Python code: `destinations = [(2, 0), (2, 3), (0, 3), (0, 0)]`

### Step counts:

- Leg 1: (0, 0) to (2, 0): **2** steps
- Leg 2: (2, 0) to (2, 3): **3** steps
- Leg 3: (2, 3) to (0, 3): **2** steps
- Leg 4: (0, 3) to (0, 0): **3** steps
- Total: **10** steps

## Part C: Hand-Trace One Leg (Example: Leg 1)

**From:** (0, 0) **to:** (2, 0)

**Computed path:** [(1, 0), (2, 0)]

Headings: 0 = North, 1 = East, 2 = South, 3 = West `turn_to` each pass: turn right, add 1 to heading, wrap 4 → 0. Stop when heading == desired.

| Step | From   | To     | Desired Heading | Current Heading | turn_to heading values | New Heading |
|------|--------|--------|-----------------|-----------------|------------------------|-------------|
| 1    | (0, 0) | (1, 0) | 2 (S)           | 0 (N)           | 0 → 1 → 2              | 2 (S)       |
| 2    | (1, 0) | (2, 0) | 2 (S)           | 2 (S)           | already there          | 2 (S)       |

### After this leg:

- New position: **(2, 0)**
- New heading: **2 (S)**
- Captured in the main program by: `position, heading = drive_path(path, position, heading)`

What heading will the robot have at the START of leg 2? **2 (S)** (the heading carries over – it's the same `heading` variable used every time through the loop)

## Part D: Testing Checklist

All items should be checked off. For Level 1 sample output:

≡≡ XRP Grid Navigation: Final Project ≡≡

Starting at: (0, 0)

Destinations: [(2, 0), (2, 3), (0, 3), (0, 0)]

--- Navigating to (2, 0) ---

Path: [(1, 0), (2, 0)]

Steps: 2

Arrived at: (2, 0)

Heading: S

--- Navigating to (2, 3) ---

Path: [(2, 1), (2, 2), (2, 3)]

Steps: 3

Arrived at: (2, 3)

Heading: E

--- Navigating to (0, 3) ---

Path: [(1, 3), (0, 3)]

Steps: 2

Arrived at: (0, 3)

Heading: N

--- Navigating to (0, 0) ---

Path: [(0, 2), (0, 1), (0, 0)]

Steps: 3

Arrived at: (0, 0)

Heading: W

≡≡ All destinations reached! ≡≡

Final position: (0, 0)

---

## Part E: Code Template

---

The completed main program:

```

# Create a Board object for wait_for_button()
board = Board.get_default_board()

# Set up starting position and heading
position = (0, 0)
heading = 0

# Define your list of 4+ destinations
destinations = [(2, 0), (2, 3), (0, 3), (0, 0)]

print("≡≡≡ XRP Grid Navigation: Final Project ≡≡≡")
print("Starting at:", position)
print("Destinations:", destinations)
print()

# Wait for button press before starting
board.wait_for_button()

# Write a for loop that goes through each destination
for dest in destinations:
    print("--- Navigating to", dest, "---")

    # Compute the path
    path = compute_manhattan_path(position, dest)

    print("Path:", path)
    print("Steps:", len(path))

    # Drive the path, capturing the new position and heading
    position, heading = drive_path(path, position, heading)

    print("Arrived at:", position)
    print("Heading:", HEADING_NAMES[heading])
    print()

print("≡≡≡ All destinations reached! ≡≡≡")
print("Final position:", position)

```

#### Blanks in order:

1. `Board.get_default_board()`
2. `(0, 0)`
3. `0`

4. `[(2, 0), (2, 3), (0, 3), (0, 0)]` (answers will vary)
  5. `board.wait_for_button()`
  6. `dest` in `destinations`
  7. `dest`
  8. `compute_manhattan_path(position, dest)`
  9. `position, heading = drive_path(path, position, heading)`
- 

## Reflection

---

### What was the hardest part of putting the whole system together?

Sample answer: The hardest part is usually remembering to reassign `position, heading = drive_path( ... )` after each leg. Without that, every path computation starts from (0, 0) regardless of where the robot actually is, so the second leg's path is wrong and the robot drives to the wrong place. The testing strategy of Level 1 (path computation only, no robot) helps catch this - you can see in the print output whether each path starts from the correct position. Another common difficulty is debugging physical robot movement: the robot may drift slightly on turns, causing it to miss grid lines. Testing with a single leg first (Level 2) before running the full sequence (Level 3) helps isolate these physical issues from logic bugs.

---

## Part F (Optional Extension): The Class Version

---

1. What line does the main program need after `navigator.drive_path(path)` that the functions version does NOT need?

```
manhattan.position = navigator.position
```

2. Why is that line a common source of bugs?

Because it requires two separate objects (`manhattan` and `navigator`) to agree on where the robot is. If a student forgets this line, `manhattan` keeps thinking the robot is still at its very first starting position, so every path after the first leg is computed from the wrong place - even though `navigator` itself is tracking correctly. The functions version only has ONE `position` variable, so there's nothing that can get out of sync.

3. Rewrite the functions-version snippet using the classes:

```
path = manhattan.compute_path(dest)
navigator.drive_path(path)
manhattan.position = navigator.position
```